

# CarKeeper

## Vehicle Maintenance & Shareable Service History

SOFTWARE ENGINEERING CAPSTONE PROJECT

Mark Enriquez | Institute of Data | October 2026

PROJECT DOCUMENTATION

## 1. Introduction

---

### Project overview

CarKeeper is a full-stack web application that helps vehicle owners organise vehicle information, record maintenance history and monitor upcoming maintenance needs. It also lets an owner generate a read-only public link to share a vehicle's service history without giving the recipient an account or editing rights.

### Problem and opportunity

Vehicle maintenance information is often scattered across paper receipts, workshop invoices, notes and personal memory. Owners may struggle to reconstruct work completed on a vehicle, identify upcoming WOF or registration deadlines, or present a coherent service history when selling a vehicle.

### Why this matters

A consolidated history supports better maintenance decisions and reduces the effort of locating old records. A shareable history can also make communication with prospective buyers or other interested parties more straightforward. CarKeeper does not independently verify workshop entries; information is supplied by the vehicle owner.

### Current state and desired state

Current state: maintenance details are fragmented and can be difficult to retrieve. Desired state: an authenticated owner can manage vehicles and service entries from one dashboard, see maintenance-related information, and selectively share read-only history through a revocable link.

### Comparable solutions

Existing applications such as CARFAX Car Care, AUTOSIST and Drivvo offer vehicle maintenance tracking, service history management and reminders. However, these platforms often include broader features such as fleet management, fuel tracking and expense reporting.

CarKeeper focuses on a simpler experience for everyday vehicle owners, combining maintenance tracking, New Zealand-specific WOF and registration expiry information, and shareable read-only service histories. A practical distinction is its owner-controlled, revocable public link, which lets recipients view a service history without creating an account. Similar applications offer forms of record sharing, so this is a focused design choice rather than an entirely new concept.

## 2. Industry / Domain

---

### Domain overview

CarKeeper sits in the automotive ownership and maintenance-management domain. It supports record keeping across servicing, WOF and vehicle registration, including information that may be useful when sharing a vehicle's history with a prospective buyer. The application is not a workshop booking, insurance or vehicle marketplace platform.

### Current industry landscape

Vehicle maintenance records are often spread across paper receipts, workshop invoices and digital platforms, making them difficult to organise and access. CarKeeper focuses on a straightforward approach to managing vehicle records, upcoming maintenance and shareable service histories.

## Domain participants

The primary participant is the vehicle owner. Secondary participants include buyers or other recipients who may view a shared service history. Mechanics and inspection providers may supply information to owners but do not have dedicated accounts in the current MVP.

## Key domain concepts

- Vehicle profile: make, model, year, registration, transmission and mileage.
- Maintenance event: service type, date, mileage, cost, workshop and notes.
- Due information: WOF expiry, registration expiry and next service mileage.
- Service history sharing: owner-enabled, token-based, read-only access.

## Relevance to other industries

CarKeeper's approach to maintenance tracking and digital record management could also apply to fleet management, machinery and equipment maintenance. These industries similarly benefit from centralised service histories, scheduled maintenance and accessible records.

## 3. Stakeholders

| Stakeholder            | Interest / expectation                                                      |
|------------------------|-----------------------------------------------------------------------------|
| Vehicle owners         | Securely manage vehicle details, service records and due information.       |
| Public-link recipients | View shared vehicle history without logging in or editing records.          |
| Developer / maintainer | Clear project structure, repeatable tests and manageable future changes.    |
| Assessors / reviewers  | Review requirements, design decisions, implementation and testing evidence. |

## 4. Product Description / Design

The solution uses a React/Vite single-page frontend, a Node.js/Express REST API and MongoDB Atlas. Authenticated endpoints use JWT-based access checks. Vehicle images are stored in the backend file system and served by Express; the database stores their paths. Public sharing uses a token-specific read-only endpoint without requiring the recipient to log in.

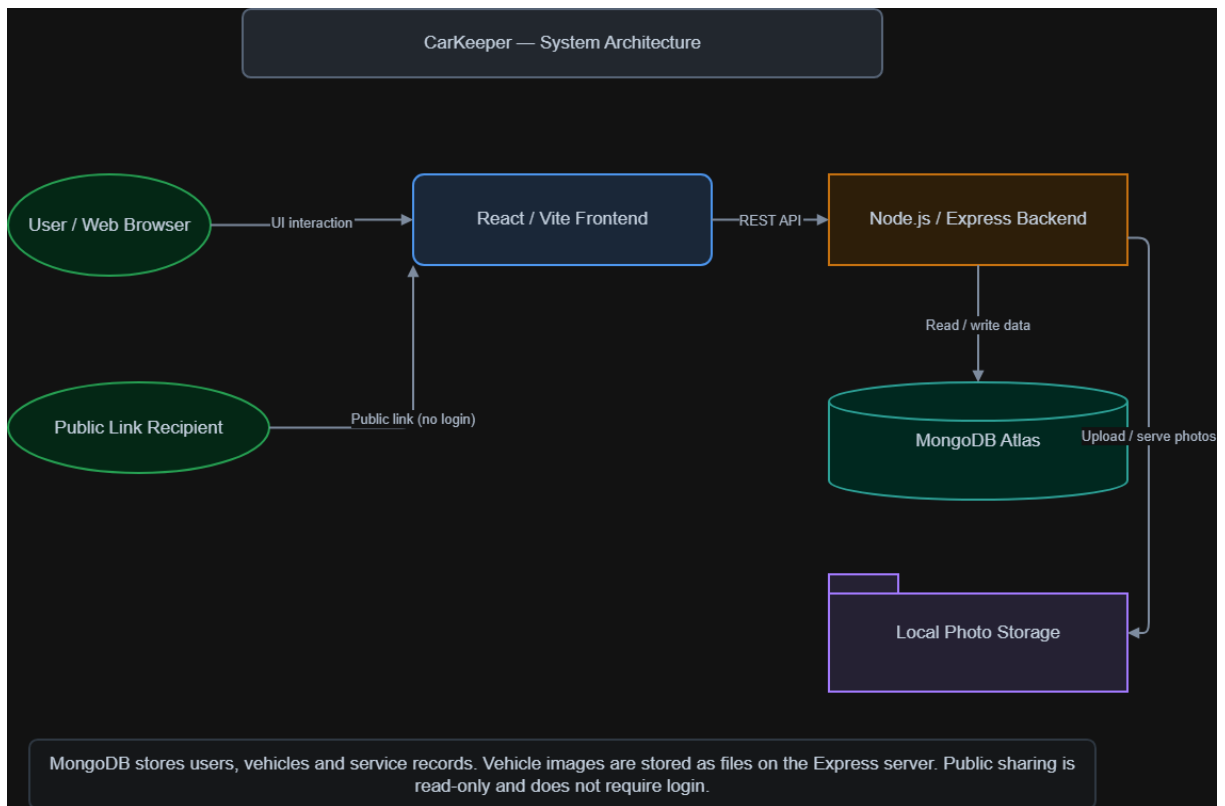


Figure 1. High-level system architecture of the CarKeeper MVP.

### Main application capabilities

- Register, log in and access protected application pages.
- Create, view, edit and delete vehicle profiles, including an optional photo.
- Create, view, edit and delete service records.
- Review dashboard summaries, upcoming maintenance and recent records.
- Enable or disable a public read-only service-history link.

## 5. User Stories for MVP 1

| ID  | User Story Title              | User Story Description                                                                               |
|-----|-------------------------------|------------------------------------------------------------------------------------------------------|
| UM1 | <b>Account Registration</b>   | As a vehicle owner, I want to create an account so that I can securely manage my vehicle records.    |
| UM2 | <b>Secure Login</b>           | As a vehicle owner, I want to log in so that only I can edit my vehicle information.                 |
| VM1 | <b>Add Vehicle</b>            | As a vehicle owner, I want to add a vehicle so that I can track its details and maintenance.         |
| VM2 | <b>Edit or Delete Vehicle</b> | As a vehicle owner, I want to edit or delete a vehicle so that my garage records remain accurate.    |
| VM3 | <b>Upload Vehicle Photo</b>   | As a vehicle owner, I want to upload a photo so that I can identify my vehicle visually.             |
| SR1 | <b>Add Service Record</b>     | As a vehicle owner, I want to record servicing and repairs so that I can maintain a service history. |

| ID  | User Story Title                     | User Story Description                                                                                                |
|-----|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| SR2 | <b>Edit or Delete Service Record</b> | As a vehicle owner, I want to correct or remove service entries so that my maintenance history stays accurate.        |
| DB1 | <b>Dashboard Overview</b>            | As a vehicle owner, I want to see vehicle summaries and upcoming maintenance so that I know what needs attention.     |
| SH1 | <b>Share Service History</b>         | As a vehicle owner, I want to share a read-only service history link so that others can view maintenance records.     |
| SH2 | <b>Disable Public Sharing</b>        | As a vehicle owner, I want to disable a shared link so that I can revoke public access.                               |
| SH3 | <b>View Shared Service History</b>   | As a recipient, I want to view a shared service history without registering so that I can review the vehicle records. |

## 6. User Flow

The user journey is divided into five linked diagrams rather than one crowded flow. Authentication leads to Dashboard; navigation links to vehicle management; vehicle details lead to service history; sharing leads to a public read-only view.

### 01 Login & Registration

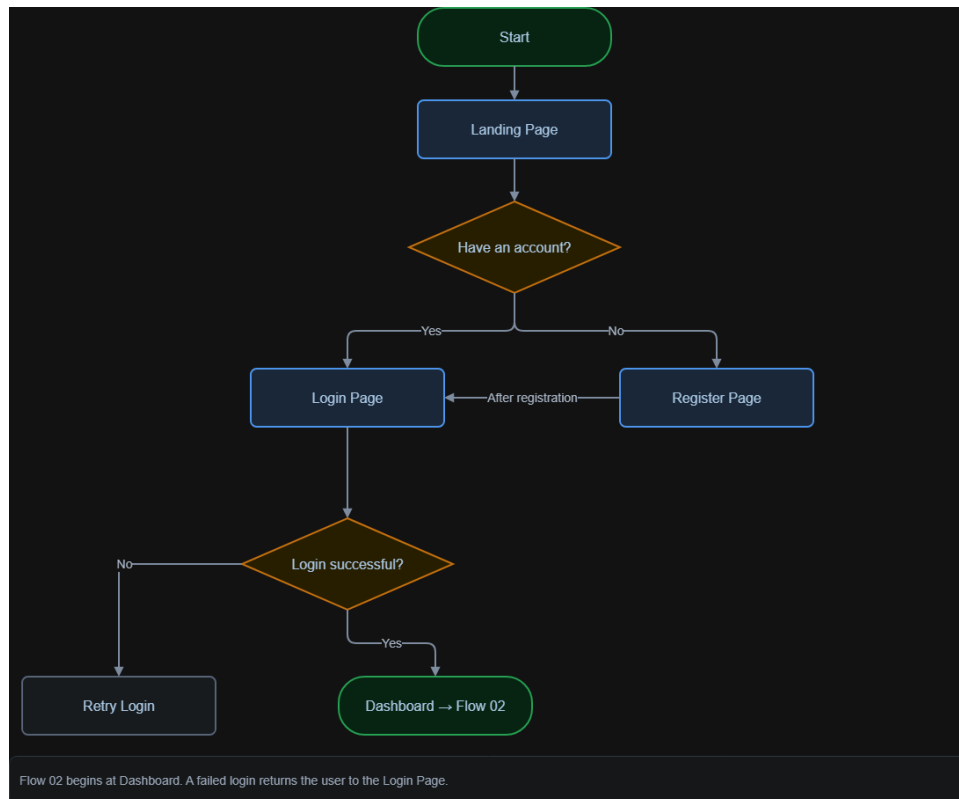


Figure 2. User flow — 01 Login & Registration

## 02 Main Navigation

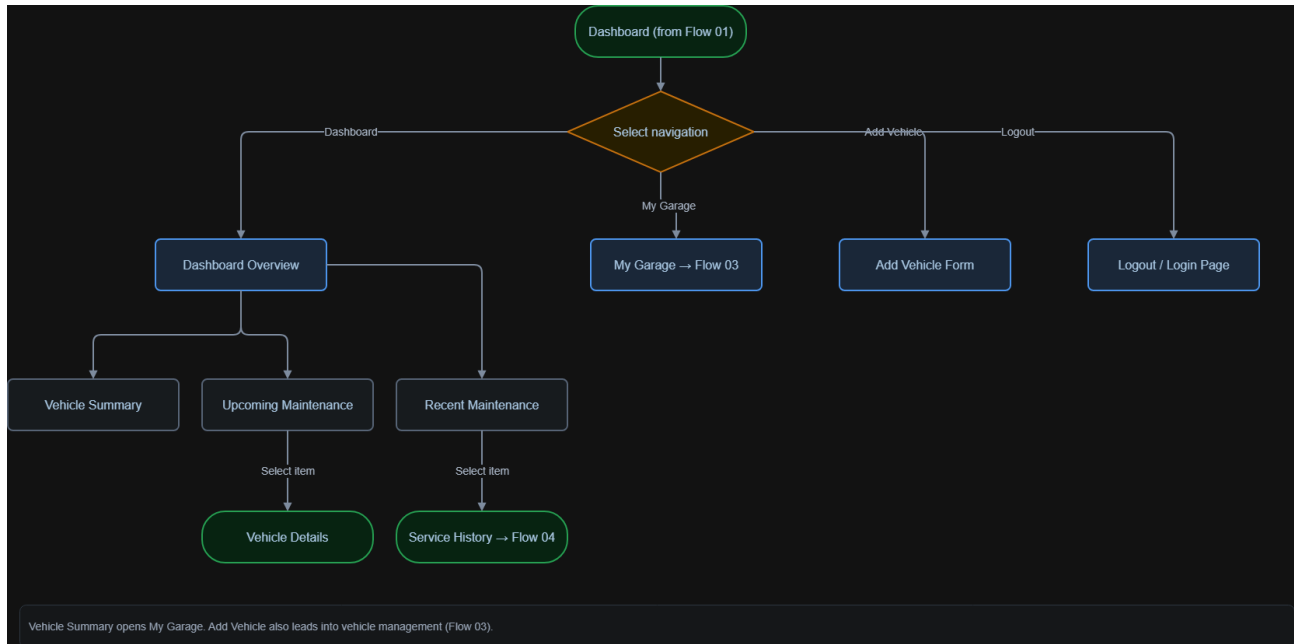


Figure 3. User flow — 02 Main Navigation

## 03 Vehicle Management

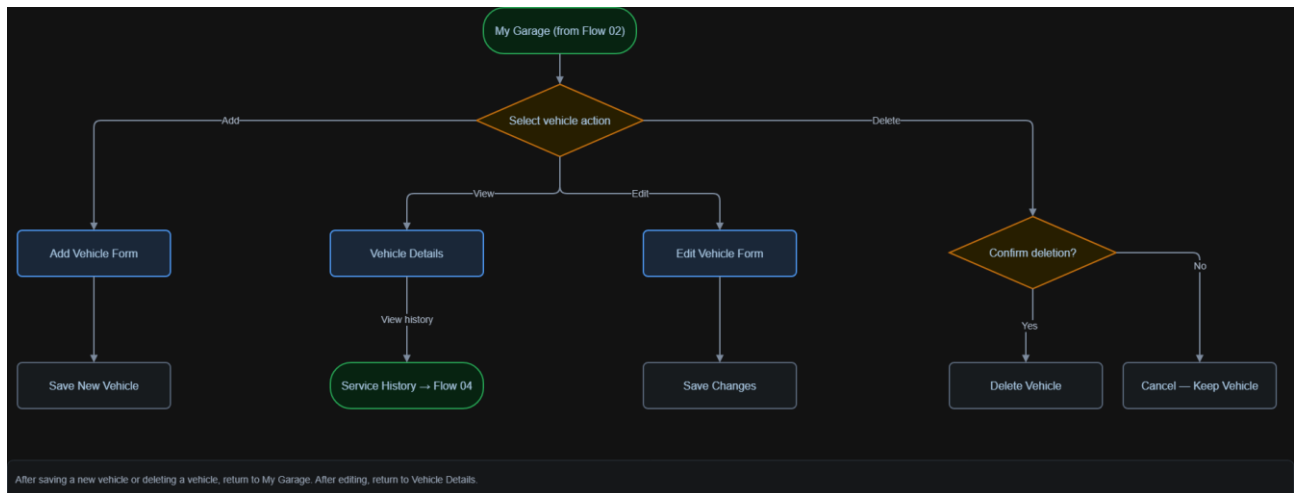


Figure 4. User flow — 03 Vehicle Management

## 04 Service History

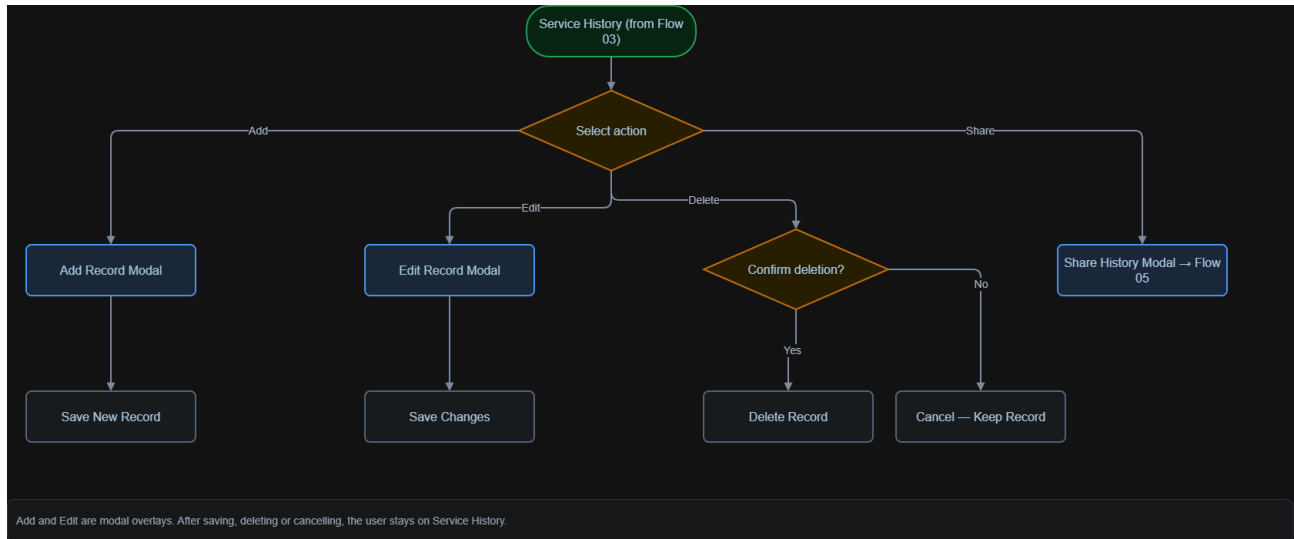


Figure 5. User flow — 04 Service History

## 05 Public Sharing

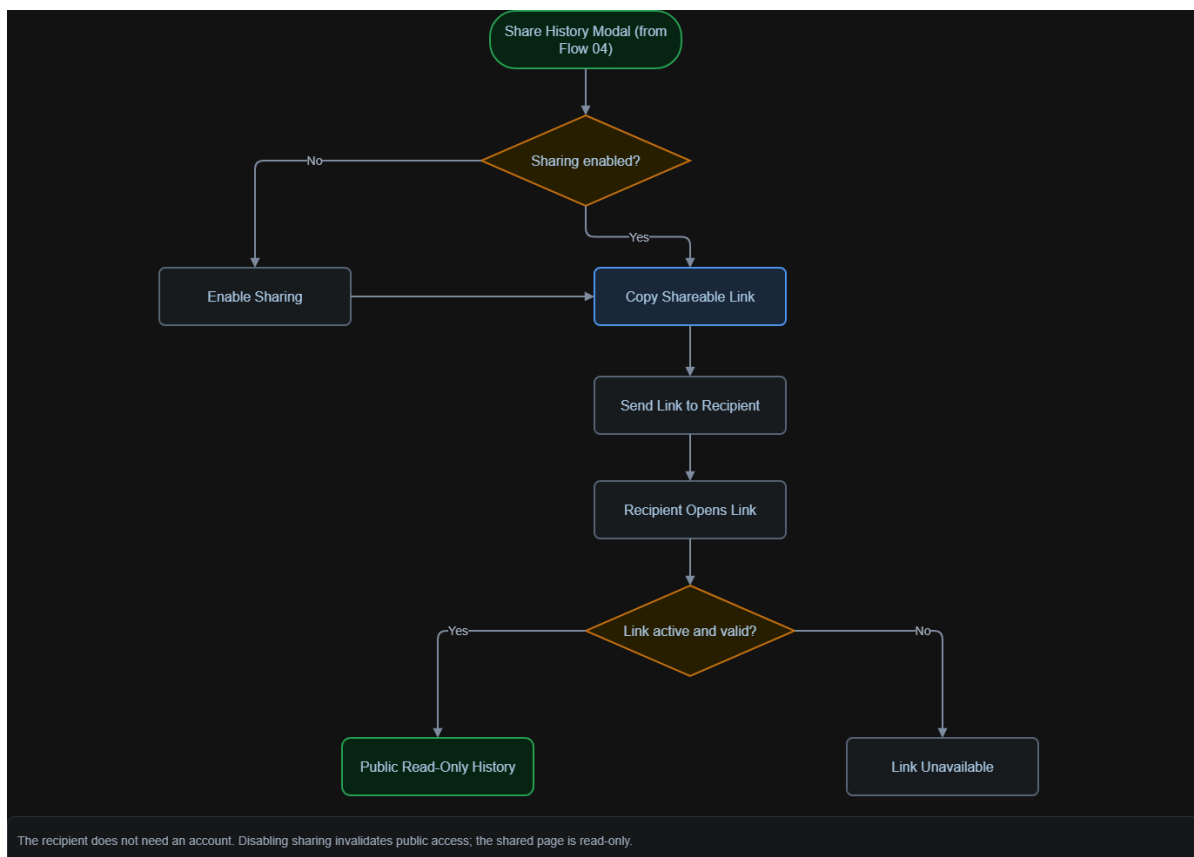


Figure 6. User flow — 05 Public Sharing

## 7. Wireframes & Design

The interface was planned in Figma using page-level wireframes for landing, registration, login, dashboard, garage, vehicle details, vehicle forms, service history and the public shared view. Add/Edit Service Record and Share History are modal overlays rather than standalone pages. The landing and authentication pages omit the logged-in navigation bar.

### Brand colour palette

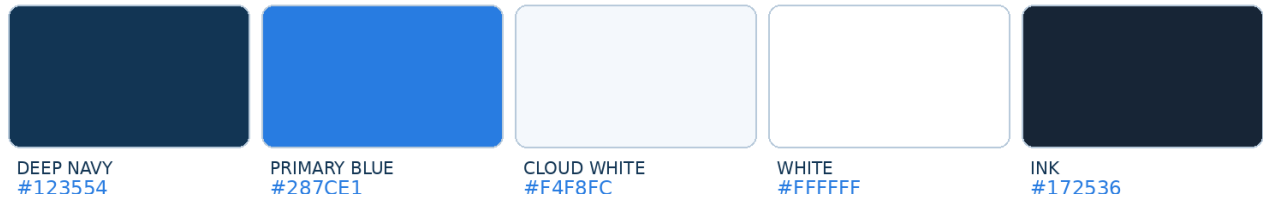


Figure 7. CarKeeper five-colour brand palette.

The palette uses restrained navy and blue accents, light surfaces and dark text to prioritise legibility. Additional status colours and borders may appear in the application but are not treated as primary brand colours.

### Wireframe evidence

CarKeeper's screen layouts were planned before implementation and refined in Figma. The layout reference below and application screenshots illustrate the intended design and implemented interface. Figma project:

<https://www.figma.com/design/n9UrRMxAyJO0N67UxRGjOY/CarKeeper?node-id=0-1>

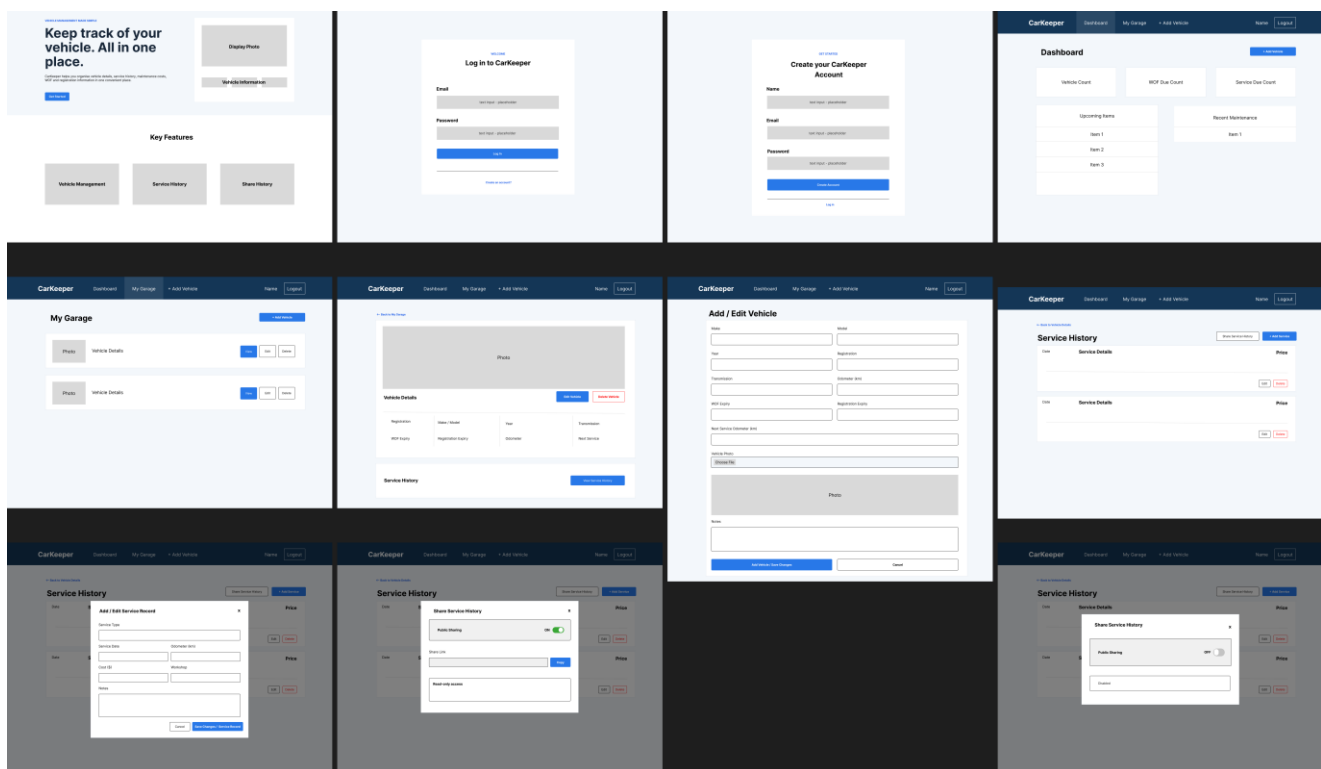


Figure 8. Initial CarKeeper layout reference showing the intended application screens (design reference).

## 8. Database Design — Logical & Physical

MongoDB Atlas holds three main collections. Each User can own multiple Vehicles; each Vehicle can contain multiple ServiceRecords. The owner and vehicle fields use Mongoose ObjectId references. The Vehicles collection also holds a photo path and an optional, sparse unique shareToken.

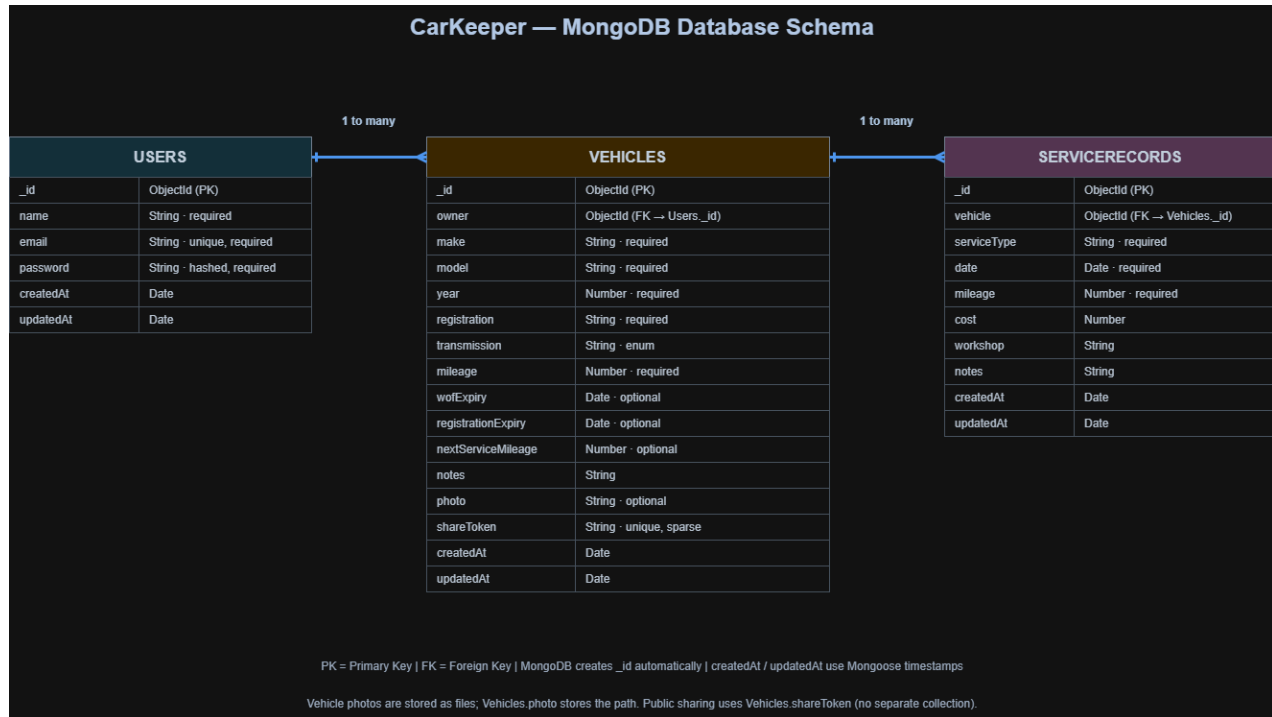


Figure 9. CarKeeper MongoDB schema showing Users, Vehicles and ServiceRecords, with one-to-many relationships and field types.

| Collection     | Core fields                                                                                                                                                      |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Users          | _id, name, email (unique), password (hashed), createdAt, updatedAt                                                                                               |
| Vehicles       | _id, owner (FK), make, model, year, registration, transmission, mileage, wofExpiry, registrationExpiry, nextServiceMileage, notes, photo, shareToken, timestamps |
| ServiceRecords | _id, vehicle (FK), serviceType, date, mileage, cost, workshop, notes, timestamps                                                                                 |

Photo files are stored under server/uploads/vehicles, rather than as MongoDB binary objects. Sharing is implemented with Vehicles.shareToken, not a separate collection.

## 9. Open Questions / Out of Scope

The following features are considered out of scope for MVP 1 and may be considered for future development:

- **Account Management & Security** – The current MVP supports user registration and login but does not include profile management, email or password changes, email verification, or forgot-password functionality. These features could improve account security and user control.
- **Receipt & Document Uploads** – Users can record maintenance details but cannot attach receipts, invoices or supporting documents to individual service records. Future development could allow users to upload documents as evidence of completed maintenance.
- **Multiple Vehicle Photos** – The current MVP supports one vehicle photo rather than a photo gallery. Future versions could allow users to upload, view and manage multiple images for each vehicle.

- **Automated Maintenance Notifications** – The dashboard displays upcoming maintenance information, but automated notifications are not implemented. Push or email notifications could remind users of upcoming servicing, WOF expiry and registration expiry dates.
- **Workshop Integration & Verified Service Records** – Maintenance records are entered by vehicle owners. The application does not provide workshop accounts or independently verified service histories. Future integration could allow mechanics to submit or verify completed work.
- **Vehicle Analytics & Third-Party Integrations** – Fuel economy tracking, advanced expense analytics, telematics and manufacturer integrations are not included. These could provide additional insight into vehicle usage and ownership costs.
- **Vehicle Marketplace & Ownership Transfer** – Although owners can share service histories, CarKeeper does not support vehicle listings, buyer accounts, or ownership transfers.
- **Cloud Hosting & File Storage** – Vehicle photos are currently stored locally on the Express server. AWS S3 could provide persistent cloud storage, while AWS Elastic Beanstalk remains a planned deployment option pending account verification.

## 10. Non-functional Requirements

| Area            | Requirement / implementation approach                                                                |
|-----------------|------------------------------------------------------------------------------------------------------|
| Security        | Use bcrypt, JWT and record-ownership checks; keep secrets out of Git.                                |
| Privacy         | Public links reveal read-only history only while sharing is enabled.                                 |
| Usability       | Clear navigation, familiar forms, feedback and modal actions.                                        |
| Data integrity  | Mongoose checks required fields and types; ObjectId references link collections.                     |
| Maintainability | Separate routes, controllers, models, middleware and reusable React components.                      |
| Performance     | Keep API responses and dashboard interactions practical for a small MVP; no benchmark claim is made. |
| Portability     | Document environment variables and run frontend/backend independently.                               |

## 11. Project Planning

Project activities were organised on a Trello board covering requirements, design, development, testing, documentation and deployment. The board records completed work and outstanding finalisation tasks. Deployment is separately marked blocked while AWS account verification is pending.

The screenshot below captures the board status at the time of documentation.

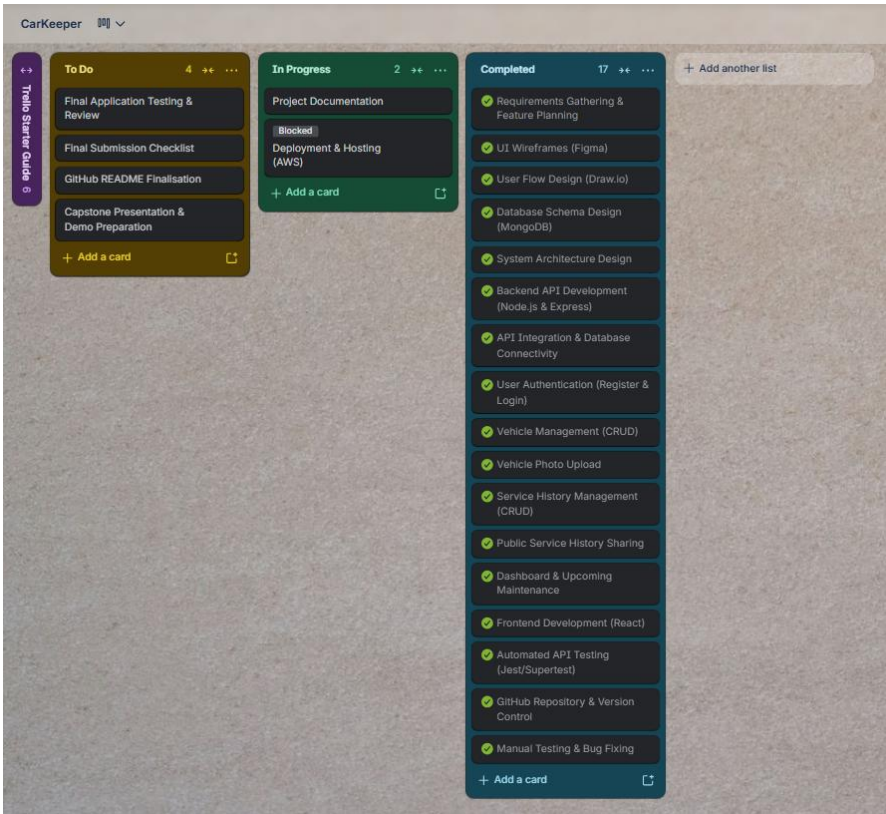


Figure 10. CarKeeper Trello planning board

## 12. Testing Strategy

### Testing approach

CarKeeper was tested incrementally as frontend and backend features were built. API integration tests provide repeatable checks of core routes, while browser-based development checks assess navigation, forms, modals, vehicle photos and the public sharing experience.

The automated tests run against a separate MongoDB test database (`carkeeper_test`), so test operations do not modify normal application records. Automated and manual evidence is reported separately below.

### Incremental development and frontend checks

After implementing a backend route, its request and response behaviour was exercised through the integration test suite. Once a React page or component was connected to the API, its workflow was checked in the browser, including routing, data display, form submissions and feedback after saving or deleting records.

The browser checks included the protected Dashboard and My Garage pages, vehicle and service-record forms, service-record modals, image handling, and read-only history opened using a public link. These development checks were not maintained as a separate signed manual test log.

### Automated backend testing — Jest and Supertest

Jest runs the integration tests, and Supertest sends HTTP requests to the Express app without requiring a separate production server process. The recorded run contains one passing suite and 19 passing tests. The cases below follow the test names shown in the captured terminal output.

| ID / test | Test steps and expected result | Result | Notes |
|-----------|--------------------------------|--------|-------|
|-----------|--------------------------------|--------|-------|

| ID / test                      | Test steps and expected result                                               | Result | Notes           |
|--------------------------------|------------------------------------------------------------------------------|--------|-----------------|
| A01 API health check           | Send GET /; confirm the API running message.                                 | Pass   | Basic endpoint  |
| A02 Register new user          | POST /api/auth/register with new user details; confirm registration.         | Pass   | Authentication  |
| A03 Reject duplicate email     | Repeat registration using an existing email; confirm rejection.              | Pass   | Validation      |
| A04 Log in successfully        | POST /api/auth/login with valid credentials; confirm successful login.       | Pass   | Authentication  |
| A05 Reject wrong password      | POST /api/auth/login with an incorrect password; confirm rejection.          | Pass   | Authentication  |
| A06 Reject missing JWT         | GET /api/vehicles without an access token; confirm refusal.                  | Pass   | Protected route |
| A07 Create vehicle             | POST /api/vehicles as an authenticated user; confirm a new vehicle.          | Pass   | Vehicle CRUD    |
| A08 List owned vehicles        | GET /api/vehicles; confirm the owner's vehicles are returned.                | Pass   | Vehicle CRUD    |
| A09 Get vehicle details        | GET /api/vehicles/:id; confirm the requested vehicle is returned.            | Pass   | Vehicle CRUD    |
| A10 Update vehicle             | PUT /api/vehicles/:id with changed details; confirm update.                  | Pass   | Vehicle CRUD    |
| A11 Add service record         | POST /api/vehicles/:vehicleId/services; confirm record creation.             | Pass   | Service CRUD    |
| A12 List service records       | GET /api/vehicles/:vehicleId/services; confirm records are returned.         | Pass   | Service CRUD    |
| A13 Update service record      | PUT /api/services/:id; confirm changed service information is saved.         | Pass   | Service CRUD    |
| A14 Delete service record      | DELETE /api/services/:id; confirm record removal.                            | Pass   | Service CRUD    |
| A15 Enable sharing             | POST /api/vehicles/:id/share; confirm public sharing is enabled.             | Pass   | Sharing         |
| A16 Public read-only retrieval | GET /api/public/vehicles/:shareToken without a JWT; confirm shared history.  | Pass   | Public access   |
| A17 Disable sharing            | DELETE /api/vehicles/:id/share; confirm sharing is disabled.                 | Pass   | Revocation      |
| A18 Reject revoked link        | GET /api/public/vehicles/:shareToken using the old token; confirm rejection. | Pass   | Revocation      |
| A19 Vehicle cleanup            | DELETE /api/vehicles/:id; confirm the test vehicle is deleted.               | Pass   | Cleanup / CRUD  |

## Automated test evidence

The screenshot below is the recorded Jest/Supertest run. It verifies 19 passed, 0 failed and one passing test suite; it does not prove that every possible validation or browser scenario has been tested.

```

PASS  __tests__/auth.test.js (7.404 s)
CarKeeper API
  ✓ GET / returns the API running message (22 ms)
Authentication API
  ✓ POST /api/auth/register registers a new user (362 ms)
  ✓ POST /api/auth/register rejects a duplicate email (48 ms)
  ✓ POST /api/auth/login logs in with valid credentials (117 ms)
  ✓ POST /api/auth/login rejects an incorrect password (112 ms)
  ✓ GET /api/vehicles rejects a request without a token (3 ms)
Vehicle API
  ✓ POST /api/vehicles creates a vehicle (64 ms)
  ✓ GET /api/vehicles returns the user's vehicles (47 ms)
  ✓ GET /api/vehicles/:id returns one vehicle (45 ms)
  ✓ PUT /api/vehicles/:id updates a vehicle (88 ms)
Service Record API
  ✓ POST /api/vehicles/:vehicleId/services creates a service record (91 ms)
  ✓ GET /api/vehicles/:vehicleId/services returns service records (83 ms)
  ✓ PUT /api/services/:id updates a service record (125 ms)
  ✓ DELETE /api/services/:id deletes a service record (124 ms)
Service History Sharing API
  ✓ POST /api/vehicles/:id/share enables sharing (83 ms)
  ✓ GET /api/public/vehicles/:shareToken returns shared history without authentication (81 ms)
  ✓ DELETE /api/vehicles/:id/share disables sharing (85 ms)
  ✓ GET /api/public/vehicles/:shareToken rejects the old token after sharing is disabled (42 ms)
Vehicle Cleanup
  ✓ DELETE /api/vehicles/:id deletes the vehicle (45 ms)

Test Suites: 1 passed, 1 total
Tests:       19 passed, 19 total
Snapshots:   0 total
Time:        10.343 s

```

Figure 11. Jest/Supertest automated API test results: 19 passed, 0 failed.

## Manual acceptance tests

The scripts below document the user journeys exercised or considered during development. “Development check” reflects the existing project record that these workflows were manually exercised; separate dated pass/fail evidence was not provided. “Final recheck” identifies a scenario to confirm before submission. These labels are deliberately distinct from the verified automated passes above.

| Test scenario                  | Steps / acceptance criteria                                                                                                                                                                                     | Outcome           | Notes        |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|--------------|
| M01 Register and log in        | <ol style="list-style-type: none"> <li>Open Register;</li> <li>Enter user details and submit.</li> <li>Log in and confirm navigation to Dashboard.</li> </ol>                                                   | Development check | Auth flow    |
| M02 Dashboard information      | <ol style="list-style-type: none"> <li>Open Dashboard with vehicles recorded;</li> <li>Inspect vehicle summary, due information and recent maintenance.</li> </ol>                                              | Final recheck     | UI display   |
| M03 Add vehicle                | <ol style="list-style-type: none"> <li>Open Add Vehicle;</li> <li>Enter required details and save.</li> <li>Confirm the new entry appears in My Garage.</li> </ol>                                              | Development check | Vehicle CRUD |
| M04 Edit vehicle               | <ol style="list-style-type: none"> <li>Open existing vehicle;</li> <li>Change details and save.</li> <li>Reopen details and verify the changes.</li> </ol>                                                      | Development check | Vehicle CRUD |
| M05 Delete vehicle             | <ol style="list-style-type: none"> <li>Select a vehicle;</li> <li>Initiate deletion and confirm.</li> <li>Check that it is removed from the garage.</li> </ol>                                                  | Development check | Vehicle CRUD |
| M06 Upload vehicle photo       | <ol style="list-style-type: none"> <li>Attach a supported photo when adding a vehicle;</li> <li>Save and confirm it displays on the vehicle card/details.</li> </ol>                                            | Development check | Image upload |
| M07 Remove photo before saving | <ol style="list-style-type: none"> <li>Open Add Vehicle and select an image;</li> <li>Remove the selected photo before submitting;</li> <li>Save and confirm the vehicle uses the no-photo fallback.</li> </ol> | Final recheck     | Add form     |

| Test scenario              | Steps / acceptance criteria                                                                                                                                                                 | Outcome           | Notes        |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|--------------|
| M08 Add service record     | <ol style="list-style-type: none"> <li>1. Open vehicle Service History;</li> <li>2. Use Add Record modal;</li> <li>3. Submit valid details;</li> <li>4. Confirm record appears.</li> </ol>  | Development check | Service CRUD |
| M09 Edit service record    | <ol style="list-style-type: none"> <li>1. Choose an existing record;</li> <li>2. Edit it in the modal;</li> <li>3. Save and confirm the updated values.</li> </ol>                          | Development check | Service CRUD |
| M10 Delete service record  | <ol style="list-style-type: none"> <li>1. Choose a service record;</li> <li>2. Delete and confirm it no longer appears in the list.</li> </ol>                                              | Development check | Service CRUD |
| M11 Enable public sharing  | <ol style="list-style-type: none"> <li>1. Enable sharing in Share History modal;</li> <li>2. Copy the URL;</li> <li>3. Open it without logging in and confirm read-only history.</li> </ol> | Development check | Public link  |
| M12 Disable public sharing | <ol style="list-style-type: none"> <li>1. Disable sharing and attempt the previous public URL again;</li> <li>2. Confirm history is unavailable.</li> </ol>                                 | Development check | Revoked link |
| M13 Protected page access  | <ol style="list-style-type: none"> <li>1. Open a protected route after removing authentication;</li> <li>2. Confirm the route is inaccessible.</li> </ol>                                   | Final recheck     | JWT API pass |

### Edge cases and test boundaries

The automated suite explicitly covers duplicate account registration, incorrect passwords, missing authentication for a protected vehicle route, and attempts to reuse a revoked share token. Browser validation, unauthorised cross-owner access, unusual image files, mobile layouts and accessibility would benefit from additional, separately recorded test cases before production use.

### Known limitations

The 19 automated passes demonstrate the tested API behaviours, not full production readiness. The current evidence does not establish load performance, cross-browser compatibility, accessibility conformance, comprehensive security testing, or deployment validation. AWS hosting was not completed because account verification was pending.

## 13. Implementation

### Frontend

React and Vite provide a single-page application. React Router separates public routes from protected owner pages. AuthContext and ProtectedRoute coordinate authentication state; Axios handles REST requests and attaches the JWT for protected endpoints. Components are organised around vehicles, service records, dashboard and layout.

### Backend

Node.js and Express expose authentication, vehicle, service-record and sharing routes. Controllers implement application logic, Mongoose models define validation and persistence, and middleware handles authentication, authorisation and vehicle-photo uploads. The architecture is one modular Express backend, not a microservices system.

### Database and storage

MongoDB Atlas stores users, vehicles and service records. The owner/vehicle references support one-to-many relationships. Uploaded photos are saved locally and served as static files by Express; MongoDB stores the corresponding path.

## Source control and hosting

The codebase is tracked in Git and pushed to GitHub. Secrets, dependencies and uploaded files are excluded from version control where appropriate. AWS Elastic Beanstalk was considered for deployment, but account verification prevented completion at the time of this document. No production deployment is claimed.

## 14. End-to-End Solution

---

CarKeeper provides a complete vehicle maintenance management experience, from account registration and vehicle management to recording maintenance history and sharing service records. After logging in, users can access their dashboard, manage vehicles through My Garage, and add or update service records using modal forms.

Users can also enable public sharing and generate a read-only link to a vehicle's service history. Recipients can access the shared records without creating an account, while vehicle owners retain control over access by enabling or disabling sharing.

### MVP Feature Summary

- **User Authentication:** Secure account registration and login using bcrypt password hashing, JWT authentication and protected routes.
- **Vehicle Management:** Create, view, update and delete vehicle profiles containing registration, make, model, year, transmission and mileage.
- **Vehicle Photo Upload:** Upload and display an optional vehicle photo.
- **Service History Management:** Create, view, update and delete maintenance records, including service type, date, mileage, cost, workshop and notes.
- **Dashboard:** Display vehicle summaries, upcoming maintenance requirements and recent service history.
- **Maintenance Tracking:** Monitor WOF expiry, vehicle registration expiry and upcoming service mileage.
- **Public Service History Sharing:** Generate a token-based, read-only link that recipients can access without logging in.
- **Sharing Access Control:** Enable or disable public sharing, invalidating previously issued links when access is revoked.

## 15. Lessons Learned

---

- **Importance of Planning and Scope Definition:** Planning helped establish a clearer understanding of the project's requirements, overall scope and complexity. Creating user stories, wireframes and diagrams provided better visibility of the work involved and helped identify dependencies before development.
- **Breaking Down User Stories:** Dividing larger user stories into smaller, manageable development tasks made implementation more structured and achievable. It also helped prioritise features, track progress and test functionality incrementally.
- **Modular Application Design:** Separating frontend components and backend routes, controllers, models and middleware improved code organisation, maintainability and troubleshooting.
- **Authentication and Security:** Implementing JWT-based authentication, protected routes, password hashing and token-based public sharing strengthened my understanding of how to secure user data and control access to application features.
- **Adaptability and Independent Learning:** Developing features beyond the course modules required independent research, experimentation and problem-solving. This reinforced the importance of being adaptable and learning unfamiliar technologies when project requirements demand it.
- **Testing and Debugging:** Automated API testing with Jest and Supertest, alongside manual frontend testing, helped identify issues, validate functionality and improve confidence in the application.
- **Deployment and External Dependencies:** The AWS account verification issue highlighted that deployment involves external dependencies beyond application development. It reinforced the importance of identifying potential blockers early and preparing alternative plans.

## 16. References

---

- Project repository: CarKeeper source code on GitHub (<https://github.com/markenrqz/CarKeeper/tree/main>).
- Figma: CarKeeper wireframes and interface planning (<https://www.figma.com/design/n9UrMXAyJO0N67UxRGjOY/CarKeeper?node-id=0-1>).
- diagrams.net (draw.io): used to create the user flows, system architecture and database schema.
- Trello: CarKeeper planning board; screenshot included in Section 11.
- Technology documentation: React, Vite, React Router, Node.js, Express, MongoDB/Mongoose, Axios, bcrypt, JWT, Multer, Jest and Supertest.